PHASE

11

Process API: fork, exec, wait and exit

Implement the UNIX process model, and use the page-fault handler to make fork cheap and memory grow on demand

Overall progress toward Turk-OS 1.0 (this phase in red) 70 % → 77 %

Duration	3 weeks
Difficulty	★★★★★ (5/5)
You need	Phase 10: user processes in their own address spaces, system calls, ELF loading, `copy_from_user`
You will have	`fork` with copy-on-write, `execve` with `argc`/`argv`, `waitpid`, `exit` with zombies and orphan reparenting, `getppid`, `brk` with lazy allocation, a stack that grows on demand, minimal signals (`kill`, `SIGSEGV`, `SIGINT`), and a user-space `init` process that starts and reaps other programs.

1 Why this phase

UNIX creates processes in an unusual way, and it turned out to be one of its best ideas. Instead of one call that starts a program, there are two. `fork` makes an exact copy of the calling process. `exec` replaces the calling process's program with a new one. The parent then uses `wait` to learn when its child has finished. OSTEP chapter 5 explains why the split is so powerful: between `fork` and `exec`, the child can rearrange its own environment (redirect its output to a file, connect a pipe, change directory) without the program it is about to run knowing anything about it. Your shell in Phase 14 depends on exactly that.

A naive `fork` copies every page of the parent, only for `exec` to throw the copy away a moment later. The fix is **copy-on-write** (COW): share the pages and copy each one only when someone writes to it. COW is your first use of a page fault as a *feature* rather than an error, and the same idea gives you lazily allocated heaps and stacks that grow on demand. After this phase, the page-fault handler is one of the most important functions in the kernel.

2 Concepts you need

2.1 The four calls and their contracts

`fork()` creates a child that is a copy of the parent: same memory contents, same registers, same open files later. It returns twice: the child's PID in the parent and 0 in the child. **`execve(path, argv, envp)`** loads a new program into the current process, keeping its PID and open files; on success it never returns. **`exit(status)`** ends the process, but it can't vanish entirely: its exit status must be kept until the parent collects it, so it becomes a **zombie**. **`waitpid(pid, &status, options)`** blocks until a child exits, collects its status and frees what is left of it. If a parent exits first, its children become **orphans** and are adopted by `init` (PID 1), whose job includes reaping them (Figure 1).

2.2 Copy-on-write

At `fork`, instead of copying the parent's pages, the kernel makes the child's page tables point to the same frames and marks every writable page *read-only* in *both* processes, with a spare PTE bit (one of the AVL bits 9–11) recording that it is really a COW page. Reading works as normal. The first write by either process causes a page fault; the handler sees the COW bit, copies that one page to a

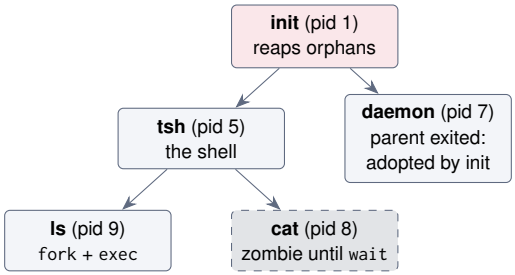


Figure 1: A Turk-OS process tree in Phase 14. Every process except init has a parent that must eventually `wait` for it.

fresh frame, maps the copy writable for the faulting process, and returns. The CPU retries the write, which now succeeds (Figure 2). A per-frame **reference count** tells the handler how many address spaces share a frame: if it is already down to one, the handler just makes the page writable again. OSC §10.3 describes COW, and OSC §10.2 describes the demand paging it builds on.

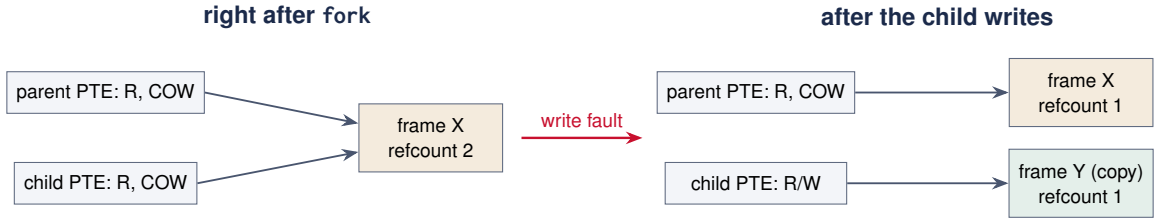


Figure 2: Copy-on-write for one page. The parent's next write finds refcount 1 and only needs its PTE made writable.

2.3 Faults as a feature

MOS §3.6.2 lists the steps of page-fault handling in a real kernel, and OSTEP chapters 21–22 go further: when memory runs out, pages can be written to disk and brought back by a fault (swapping), which needs a replacement policy such as the clock algorithm. Turk-OS uses faults for three things in this phase. **COW**: a write to a present page marked COW. **Lazy heap**: `brk` only moves the heap's end, and a fault inside the heap maps a zeroed page. **Stack growth**: a fault just below the stack maps a new page, up to a limit. Anything else from user mode is a real error and ends the process with `SIGSEGV`. Swapping is a stretch goal once you have a disk.

3 Reading plan

Priority	Book	What to read	Pages	Why now
REQUIRED	OSTEP	Ch. 5 *Interlude: Process API* (`fork`, `wait`, `exec`, why the split, redirection). Do its homework on Linux first	37–48	The exact semantics you must reproduce.
REQUIRED	OSDI	§4.7 *Overview of the MINIX 3 Process Manager*: memory layout, `fork`/`exit`/`wait` (§4.7.4), `exec` (§4.7.5), `brk` (§4.7.6), signals (§4.7.7)	420–447	A complete UNIX process model, explained step by step.
REQUIRED	OSDI	§4.8.3–4.8.6 *Implementation of fork, exit, wait, exec, brk, signal handling*	455–471	The real code for the same calls.

Priority	Book	What to read	Pages	Why now
REQUIRED	OSC	§10.2 *Demand Paging*; §10.3 *Copy-on-Write*	392–401	Page faults as the mechanism behind COW and lazy allocation.
RECOMMENDED	OSC	§3.3 *Operations on Processes*	116–123	Creation, termination, zombies and orphans.
RECOMMENDED	MOS	§3.6 *Implementation Issues* (page fault handling, instruction backup, locking pages, backing store)	233–240	What a page-fault handler really has to deal with.
RECOMMENDED	OSTEP	Ch. 21–22 *Beyond Physical Memory: Mechanisms*, *Policies*	227–254	Swapping and replacement policies, for the stretch goal.
OPTIONAL	MOS	§3.4 *Page Replacement Algorithms*	209–222	Optimal, NRU, FIFO, second chance, clock, LRU, working set.
OPTIONAL	MOS	§10.3 *Processes in Linux*	733–753	`fork`, `clone` and `exec` in a production kernel.
RECOMMENDED	LKD	Ch. 3, *Process Creation*, *The Linux Implementation of Threads*, *Process Termination*	31–40	Copy-on-write, `fork` and zombies as Linux does them.
RECOMMENDED	LKD	Ch. 15, *The Process Address Space*	305–322	Memory descriptors and memory areas: where `brk` and the stack live.

Real-system lens: how MINIX 3 does it

The MINIX process manager is a user-space server, and its handlers read almost like pseudo-code: `do_fork`, `do_pm_exit` and `do_waitpid` in `servers/pm/forkexit.c (18400)`, `do_exec` in `servers/pm/exec.c (18700)`, `do_brk` in `servers/pm/break.c (19300)`, and signal handling in `servers/pm/signal.c (19500)`. Notice how `exec` builds the new stack with the argument strings and then patches the pointers in it: you will do the same in Task 11.6. One difference is fundamental: MINIX 3 has no paging, so its `fork` must copy the child's whole data segment immediately. With paging you can share and copy lazily.

Linux, for comparison. LKD In Linux, `fork` is one case of `clone`: copy-on-write makes it cheap, and a thread is simply a task that shares its address space, files and signal handlers with its parent.

4 Build it

Task 11.1 Processes, parents and the trap frame

Extend `struct task` with `ppid`, a list of children, `exit_code`, a wait queue for "one of my children exited", `heap_start` and `brk`, and `tf`: a pointer to the `struct regs` that the current system call entered with. Set `current->tf = r` at the top of the syscall handler; `fork`, `exec` and signals all need to read or change the saved user registers. Put a limit on the number of processes (say 64), so that a fork bomb fails with `-EAGAIN` instead of exhausting memory.

Task 11.2 Frame reference counts

Add a reference count per physical frame to the PMM: an array of `refcount_t` from Task 9.8 covers the 512 MiB direct map with 512 KiB. `pmm_ref(pa)` calls `refcount_inc`; `pmm_unref(pa)` frees the frame when `refcount_dec_and_test` says it was the last reference. From now on, freeing a user page means unreferencing it, and a copy-on-write bug that drops one reference too many leaks a frame instead of handing it to another process.

Task 11.3 fork, the simple version

Get `fork` right before making it fast. Create the child task and a new address space, and for every present user page in the parent, allocate a frame, copy the page through the direct map, and map it at the same address with the same flags. The difficult part is returning to user mode in the child. Copy the parent's trap frame to the top of the child's kernel stack and set its EAX to 0. Below it, build a `switch_context` frame whose return address is the exit path of `isr_common`, the instructions after `call isr_handler`. Give that point a label, `isr_return`. When the child is first scheduled, it "returns" through that path and `irets` into user mode, at the instruction after `int 0x80`, with EAX = 0.

kernel/proc/fork.c (the tricky part)

```
extern void isr_return(void);              /* label in isr.s, just after "add esp, 4" */

static void setup_child_stack(struct task *child, const struct regs *parent_tf)
{
    char *top = (char *)child->kstack + KSTACK_SIZE;
    struct regs *tf = (struct regs *)(top - sizeof(struct regs));
    *tf = *parent_tf;                       /* same user registers as the parent ...  */
    tf->eax = 0;                            /* ... except fork() returns 0 in the child */

    uint32_t *sp = (uint32_t *)tf;
    *--sp = (uint32_t)isr_return;           /* switch_context's ret goes here */
    *--sp = 0;                              /* ebp */
    *--sp = 0;                              /* ebx */
    *--sp = 0;                              /* esi */
    *--sp = 0;                              /* edi */
    child->esp = (uint32_t)sp;
    child->tf  = tf;
}
```

Verify: A test where the parent sets a variable to 1 and the child sets it to 2 shows each process keeping its own value, and `fork` returns the child's PID in the parent.

Task 11.4 Copy-on-write

Replace the copying loop. For every present user PTE in the parent: if it is writable, clear R/W and set your `PG_COW` bit (`0x200`) in the parent's PTE; copy the PTE into the child; call `pmm_ref` on the frame. When the loop is done, reload CR3 in the parent: you changed many of its live mappings, and its TLB still holds the old writable ones. Then extend the page-fault handler.

kernel/mm/fault.c (COW case)

```
/* A write to a present page (error bits P and W set) below the kernel. The fault can come
 * from user mode or from the kernel writing through copy_to_user (CR0.WP is set). */
if ((r->err_code & 3) == 3 && addr < KERNEL_VMA) {
    pte_t *pte = vmm_get_pte(current->pgdir, addr);
    if (pte && (*pte & PG_COW)) {
        uintptr_t old = *pte & ~0xFFFu;
        if (pmm_refcount(old) == 1) {                   /* last user: just take it back */
            *pte = (*pte | PG_WRITE) & ~PG_COW;
        } else {
            uintptr_t copy = pmm_alloc_frame();
            if (!copy) { kill_current(SIGKILL); return; }
            memcpy(P2V(copy), P2V(old), PAGE_SIZE);
            *pte = copy | (*pte & 0xFFF & ~PG_COW) | PG_WRITE;
            pmm_unref(old);
        }
        __asm__ volatile ("invlpg (%0)" : : "r"(addr) : "memory");
        return;                                         /* the CPU retries the write */
    }
}
```

Verify: Right after forking a process with a 1 MiB array, the PMM free count drops only by a few frames (page tables and the kernel stack). Each write to a new page of the array in the child costs exactly one more frame.

Task 11.5 exit and waitpid

`sys_exit` frees the user pages and page tables (unreferencing frames), gives every child to `init`, marks the task as a zombie with its exit code, wakes the parent's wait queue, and schedules away. Keep the page directory itself and the kernel stack for the parent to free: the exiting task is still running on both. `sys_waitpid` looks for a zombie child that matches (`pid == -1` means any): if it finds one, it copies the status out with `copy_to_user`, frees the child completely and returns its PID. If there are no children at all it returns `-ECHILD`; with `WNOHANG` it returns 0; otherwise it sleeps on its wait queue and looks again.

Verify: 1000 iterations of fork, exit and wait bring the PMM free count and the heap statistics back to exactly their starting values. A child whose parent exits first is reaped by `init`.

Task 11.6 execve with arguments

The arguments live in the old address space, which you are about to throw away, so copy the path and the `argv/envp` strings into kernel buffers first (with size limits). Build a complete new address space and load the ELF into it. Only when everything has succeeded, switch to it, free the old one, and edit the trap frame: EIP is the new entry point, the user ESP is the new stack pointer. That way a failed `exec` returns an error to a process that still works. Lay out the new stack the way the System V i386 ABI does, and update `crt0`.

initial user stack built by execve (higher addresses at the top)

```
0xC0000000  +-----------------------------------+
            | argument and environment strings  |  "hello\0" "-v\0" "PATH=/bin\0"
            +-----------------------------------+
            | NULL                              |
            | envp[0]                           |  pointers into the strings above
            | NULL                              |
            | argv[1], argv[0]                  |
ESP  -->    | argc                              |  _start reads this first
            +-----------------------------------+
```

user/crt0.s (new version)

```
_start:
    mov  eax, [esp]             ; argc
    lea  ebx, [esp + 4]         ; argv
    lea  ecx, [ebx + eax*4 + 4] ; envp = argv + argc + 1
    push ecx
    push ebx
    push eax
    call main                   ; main(argc, argv, envp)
    push eax
    call exit
```

Until Phase 13 brings a file system, `execve` finds programs by name in the table of boot modules, so `execve("/bin/hello", ...)` looks up the module called `hello`.

Verify: A program that prints its arguments shows exactly what its parent passed to `execve`. A failing `execve` (unknown program) returns `-ENOENT` and the caller continues normally.

Task 11.7 brk and a growing stack

`sys_brk(addr)` returns the current end of the heap when `addr` is 0, and otherwise moves the end, refusing to grow into the stack region; when the heap shrinks, unmap and unreference the pages beyond the new end. Don't map anything when it grows: in the page-fault handler, a not-present fault inside `[heap_start, brk)` maps a zeroed page. Similarly, a fault less than 8 MiB below `0xC0000000` and near the current stack maps a new stack page.

Verify: A user program that `brk`s 64 MiB but touches only three pages costs three frames.

Task 11.8 Minimal signals

Add a pending-signal bitmask to each task, and `sys_kill(pid, sig)`. Before returning to user mode (in `isr_handler`, when the saved CS has RPL 3), check pending signals and apply the default action; for now every signal Turk-OS supports terminates the process, with exit status $128 +$ signal so the parent can tell. Turn user-mode exceptions into signals (`SIGSEGV` for page faults and #GP, `SIGFPE` for divide errors, `SIGILL` for invalid opcodes). Make Ctrl+C on the console send `SIGINT` to a designated foreground process; Phase 14 formalises this with the terminal driver.

Task 11.9 init, and the tests

Write `user/init.c`: the kernel starts it as PID 1 once the boot is done. For now it forks and execs the test programs one after another and loops on `waitpid(-1, ...)` forever, which also reaps orphans. Add these tests to `make test`.

Test	What must hold
`fork_values`	parent and child see different return values and keep separate copies of variables
`cow_frames`	forking a large process costs a few frames; each written page costs exactly one more
`fork_exec_wait`	a child execs a program that exits with 42; `waitpid` reports 42
`orphans`	grandchildren of an exited process are reaped by init; no zombies remain
`no_leaks`	1000 fork/exec/exit/wait cycles leave PMM and heap statistics unchanged
`fork_bomb`	forking without limit ends with `-EAGAIN`, and the system recovers when the processes exit
`signals`	`kill(child, SIGKILL)` ends a looping child; `waitpid` reports status $128 + 9$

5 Testing and debugging

Symptom	Likely cause	What to do
The child crashes as soon as it runs	Its kernel stack doesn't match what `isr_return` pops, or the trap frame wasn't copied	Check in GDB that ESP at `isr_return` equals the address of the child's `struct regs`.
Parent and child see each other's writes after `fork`	The parent's TLB still holds writable entries for pages now marked COW	Reload CR3 in the parent after the COW loop.
A write by the kernel to user memory corrupts another process	CR0.WP not set, so the kernel wrote straight through a COW page	Set CR0.WP (Phase 6); the fault handler then copies the page for the kernel too.
Free memory shrinks a little with every fork	A reference count isn't dropped somewhere: exit, exec's old address space, or a COW copy	Run `no_leaks`; print the counts before and after one cycle.
Zombies pile up	Nobody waits: the shell or init doesn't reap	init loops on `waitpid(-1)`; later the shell reaps background jobs.
A failed `exec` leaves a broken process	The old address space was freed before the load succeeded	Build first, swap last.

6 Milestone

Milestone: The UNIX process model

- ☐ `fork` with copy-on-write, `execve` with arguments, `waitpid`, `exit`, `getpid`, `getppid` and `brk` work.
- ☐ The page-fault handler resolves COW, lazy heap and stack growth, and turns everything else into `SIGSEGV`.
- ☐ `init` runs as PID 1, starts programs and reaps every orphan.
- ☐ All seven tests pass, including 1000 cycles without leaks.

Self-check questions

1. Why does UNIX separate `fork` and `exec`? Give an example the split makes easy.
2. How does one call to `fork` return two different values? Point to the line in your kernel that does it.
3. Walk through copy-on-write for one page, including every TLB flush and reference count change.
4. What is a zombie, what is an orphan, and who cleans each up?
5. Why must `execve` copy its arguments into the kernel before building the new address space?
6. What does `exec` keep from the old process, and what does it replace?
7. Why does a lazy `brk` make programs that over-allocate cheap? What is the risk?

7 Going further

Once Phase 12 gives you a disk, add swapping: when the PMM runs low, pick a victim page with the clock algorithm (OSTEP chapter 22, MOS §3.4.5), write it to a swap partition, mark the PTE not present with the disk slot in the free bits, and read it back on the next fault. Add user-level signal handlers with `sigaction` and a `sigreturn` trampoline on the user stack. Or implement threads inside a process with a `clone`-style call that shares the address space.

Next: **Phase 12 Storage: Block Devices, ATA Driver and Buffer Cache** →

Books used in this plan

LOB — Erik Helin, Adam Renberg. *The little book about OS development*, 2015. The hands-on x86 guide. Free at https://littleosbook.github.io.

OSTEP — Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*, v0.90, Arpaci-Dusseau Books, 2015. Main theory text. Free at https://pages.cs.wisc.edu/~remzi/OSTEP/.

OSC — A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed., Wiley, 2018. Reference theory text.

MOS — Andrew S. Tanenbaum, Herbert Bos. *Modern Operating Systems*, 4th ed., Pearson, 2015. Hardware, I/O, design, security.

OSDI — Andrew S. Tanenbaum, Albert S. Woodhull. *Operating Systems: Design and Implementation*, 3rd ed., Pearson Prentice Hall, 2006. MINIX 3 with full source listing.

LKD — Robert Love. *Linux Kernel Development*, 3rd ed., Addison-Wesley, 2010. How Linux implements each subsystem.

LKP — Kaiwan N. Billimoria. *Linux Kernel Programming*, Packt, 2021. Modern kernel practice: logging, allocators, the scheduler, synchronisation.

The books do not come with this plan. LOB and OSTEP are free online; buy or borrow the other five (the roadmap lists their ISBNs). Where books disagree, the plan follows the newer one when its method is also better for Turk-OS. Page numbers are the *printed* page numbers of these editions, not PDF page indices. With another edition, or with the free OSTEP chapters (each one starts at page 1), find the reading by its chapter and section numbers. MINIX source references like `kernel/proc.c (07400)` give the file and its starting line in OSDI Appendix B.